

[BE] 우테코 7기 최종 코테

최종 코딩 테스트 진행 방식

진행 방식

- 미션은 과제 진행 요구 사항, 기능 요구 사항, 프로그래밍 요구 사항 세 가지로 구성되어 있다.
- 세 개의 요구 사항을 만족하기 위해 노력한다. 특히 기능을 구현하기 전에 기능 목록을 만들고, 기능 단위로 커밋 하는 방식으로 진행한다.
- 기능 요구 사항에 기재되지 않은 내용은 스스로 판단하여 구현한다.
- 오후 6시 전까지 구현을 완료하여 제출해야 한다. 제출은 오후 5시부터 가능하다.
 - 정해진 시간을 지키지 않을 경우 미션을 제출하지 않은 것으로 간주한다.
 - 정해진 시간 내에 우아한테크코스 계정을 협력자로 초대하지 않으면 제출하지 않은 것으로 간주한다.
 - 종료 일시 이후에는 추가 푸시를 허용하지 않는다.

미션 제출 방법

- 미션 구현을 완료한 후 GitHub을 통해 제출해야 한다.
 - GitHub을 활용한 제출 방법은 프리코스 과제 제출 문서를 참고해 제출한다.
- GitHub에 미션을 제출한 후 우아한테크코스 지원 플랫폼에서 과제 제출물을 최종 제출한다.
 - 자세한 안내는 제출 가이드를 참고한다.
 - 과제를 수행하면서 느낀 점, 배운 점, 많은 시간을 투자한 부분 등 자유롭게 작성한다.

과제 제출 전 체크 리스트

- 기능을 올바르게 구현했더라도 **요구 사항에 명시된 출력 형식을 따르지 않으면 0점**을 받게 된다.

- 기능 구현을 완료한 후 아래 가이드에 따라 모든 테스트가 성공적으로 실행되는지 확인한다.
- 테스트가 실패하면 점수가 0점이 되므로 제출하기 전에 반드시 확인한다.

테스트 실행 가이드

- 터미널에서 `java -version` 을 실행하여 Java 버전이 21인지 확인한다. Eclipse 또는 IntelliJ IDEA와 같은 IDE에서 Java 21로 실행되는지 확인한다.
- 터미널에서 Mac 또는 Linux 사용자의 경우 `./gradlew clean test` 명령을 실행하고, Windows 사용자의 경우 `gradlew.bat clean test` 또는 `.\gradlew.bat clean test` 명령을 실행할 때 모든 테스트가 아래와 같이 통과하는지 확인한다.

```
BUILD SUCCESSFUL in 0s
```

우아한테크코스 출석

과제 진행 요구 사항

- 미션은 우아한테크코스 출석 저장소를 만드는 것으로 시작한다.
- 기능을 구현하기 전 `README.md` 에 구현할 기능 목록을 정리해 추가한다.
- Git의 커밋 단위는 앞 단계에서 `README.md` 에 정리한 기능 목록 단위로 추가한다.
 - AngularJS Git Commit Message Conventions을 참고해 커밋 메시지를 작성한다.
- 자세한 과제 진행 방법은 프리코스 진행 가이드 문서를 참고한다.

기능 요구 사항

우아한테크코스에서는 학생(이하 크루)의 출석 관리에 특별한 주의를 기울이고 있습니다. 하지만, 지금까지 수동으로 관리하던 출석부로는 많은 크루의 출석 현황을 효율적으로 파악하기 어려워졌습니다. 특히, 지난달에는 출석 관리의 어려움으로 인해 여러 가지 문제가 발생

했습니다. 제때 누적 지각 크루를 파악하지 못해 면담이 지연되었고, 결석 일수를 계산하는 과정에서 실수가 발생하기도 하였습니다.

따라서 2024년 12월 한 달 동안 시범적으로 최소한의 기능을 갖춘 출석 시스템을 개발하여 출석을 체계적으로 관리하기로 하였습니다. 이 시스템은 코치가 사용하게 됩니다. 아래 요구 사항을 충족하는 출석 시스템을 개발해 주시기를 바랍니다.

출석 관리 규칙 및 시스템 설계 정책

2024.12						
일	월	화	수	목	금	토
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25 성탄절	26	27	28
29	30	31	1	2	3	4

- 출석 시스템의 출석 확인은 크루가 캠퍼스에 들어온 후 시스템에 출석 데이터가 저장된 시간을 기준으로 한다.
- 시간은 24시간 형식만 사용한다. 예를 들어, "22:30"은 오후 10시 30분을 의미한다.
- 교육 시간은 월요일은 13:00~18:00, 화요일~금요일은 10:00~18:00이다.
 - 해당 요일의 시작 시각으로부터 5분 초과는 지각으로 간주한다.
 - 해당 요일의 시작 시각으로부터 30분 초과는 결석으로 간주한다.
 - 등교하지 않아 출석 기록이 없는 날은 결석으로 간주한다.

- 누적 지각 및 결석 횟수에 따라 경고 또는 면담을 시행한다. 또한 결석 횟수가 5회를 초과할 때 제적을 시행한다.
 - 지각 3회는 결석 1회로 간주한다.
 - 경고 대상자: 결석 2회 이상
 - 면담 대상자: 결석 3회 이상
 - 제적 대상자: 결석 5회 초과
- 캠퍼스 운영 시간은 매일 08:00~23:00이다.
- 주말 및 공휴일에는 출석을 받지 않는다.
- 출석 시스템에 등록된 크루와 12월 출석 기록은 제공된 파일(attendances.csv)에서 확인할 수 있다.
- 프로그램은 사용자가 종료할 때까지 종료되지 않으며, 해당 기능을 수행한 후 초기 화면으로 돌아간다.
 - 사용자가 잘못된 값을 입력할 경우 "[ERROR]"로 시작하는 메시지와 함께 `IllegalArgumentException` 을 발생시킨 후 애플리케이션은 종료되어야 한다.

출석 확인

- 닉네임과 등교 시간을 입력하면 출석할 수 있다.
- 출석 후 출석 기록을 확인할 수 있다.
- 이미 출석한 경우, 다시 출석할 수 없으며 수정 기능을 이용하도록 안내한다.

닉네임을 입력해 주세요.

이든

등교 시간을 입력해 주세요.

09:59

12월 05일 화요일 09:59 (출석)

출석 수정

- 출석 확인을 수정하려면 닉네임, 수정하려는 날짜, 등교 시간을 입력하여 기록을 수정할 수 있다.

- 수정 후에는 변경 전과 변경 후의 출석 기록을 확인할 수 있다.

출석을 수정하려는 크루의 닉네임을 입력해 주세요.

빙티

수정하려는 날짜(일)를 입력해 주세요.

3

언제로 변경하겠습니까?

09:58

12월 03일 화요일 10:07 (지각) -> 09:58 (출석) 수정 완료!

크루별 출석 기록 확인

- 닉네임을 입력하면 전날까지의 크루 출석 기록을 확인할 수 있다.

닉네임을 입력해 주세요.

빙티

이번 달 빙티의 출석 기록입니다.

12월 02일 월요일 13:00 (출석)

12월 03일 화요일 10:07 (지각)

12월 04일 수요일 10:02 (출석)

12월 05일 목요일 10:06 (지각)

12월 06일 금요일 10:01 (출석)

12월 09일 월요일 --:-- (결석)

12월 10일 화요일 10:03 (출석)

12월 11일 수요일 --:-- (결석)

12월 12일 목요일 --:-- (결석)

12월 13일 금요일 10:02 (출석)

출석: 3회

지각: 0회

결석: 3회

면담 대상자입니다.

제적 위험자 확인

- 전날까지의 크루 출석 기록을 바탕으로 제적 위험자를 파악한다.
- 제적 위험자는 제적 대상자, 면담 대상자, 경고 대상자순으로 출력하며, 대상 항목별 정렬 순서는 지각을 결석으로 간주하여 내림차순한다. 출석 상태가 같으면 닉네임으로 오름차순 정렬한다.

제적 위험자 조회 결과

- 빙티: 결석 3회, 지각 4회 (면담)
- 이든: 결석 2회, 지각 5회 (면담)
- 빙봉: 결석 1회, 지각 6회 (면담)
- 쿠키: 결석 2회, 지각 3회 (면담)
- 짱수: 결석 0회, 지각 6회 (경고)

입출력 요구 사항

입력

- 자세한 입력 예시는 실행 결과 예시를 참고하며, 프로그램을 시작하면 `src/main/resources/attendances.csv` 를 통해 구현에 필요한 정보를 조회한다.
 - 닉네임과 출석 일시가 기록되어 있다.
 - **파일의 내용은 수정할 수 없다.**

```
nickname,datetime
쿠키,2024-12-13 10:08
빙봉,2024-12-13 10:07
빙티,2024-12-13 10:07
이든,2024-12-13 10:07
```

출력

자세한 출력 예시는 실행 결과 예시를 참고하며, 각 상황에 대한 예외 메시지는 아래와 같다.

- 기능 선택 항목, 날짜 또는 시간을 잘못된 형식으로 입력한 경우

[ERROR] 잘못된 형식을 입력하였습니다.

- 등록되지 않은 닉네임을 입력한 경우

[ERROR] 등록되지 않은 닉네임입니다.

- 주말 또는 공휴일에 출석을 확인하거나 수정하는 경우

[ERROR] 12월 14일 토요일은 등교일이 아닙니다.

- 미래 날짜로 출석을 수정하는 경우

[ERROR] 아직 수정할 수 없습니다.

- 등교 시간이 캠퍼스 운영 시간이 아닌 경우

[ERROR] 캠퍼스 운영 시간에만 출석이 가능합니다.

- 이미 출석을 하였는데 다시 출석 확인을 하는 경우

[ERROR] 이미 출석을 확인하였습니다. 필요한 경우 수정 기능을 이용해 주세요.

실행 결과 예시 1

오늘은 12월 14일 토요일입니다. 기능을 선택해 주세요.

1. 출석 확인
 2. 출석 수정
 3. 크루별 출석 기록 확인
 4. 제적 위험자 확인
 - Q. 종료
- 1

[ERROR] 12월 14일 토요일은 등교일이 아닙니다.

실행 결과 예시 2

오늘은 12월 13일 금요일입니다. 기능을 선택해 주세요.

1. 출석 확인
2. 출석 수정
3. 크루별 출석 기록 확인
4. 제적 위험자 확인
- Q. 종료

1

닉네임을 입력해 주세요.

빈봉

[ERROR] 등록되지 않은 닉네임입니다.

실행 결과 예시 3

오늘은 12월 13일 금요일입니다. 기능을 선택해 주세요.

1. 출석 확인
2. 출석 수정
3. 크루별 출석 기록 확인
4. 제적 위험자 확인
- Q. 종료

1

닉네임을 입력해 주세요.

이든

등교 시간을 입력해 주세요.

09:59

12월 13일 금요일 09:59 (출석)

오늘은 12월 13일 금요일입니다. 기능을 선택해 주세요.

1. 출석 확인
 2. 출석 수정
 3. 크루별 출석 기록 확인
 4. 제적 위험자 확인
- Q. 종료
- 2

출석을 수정하려는 크루의 닉네임을 입력해 주세요.

빙티

수정하려는 날짜(일)를 입력해 주세요.

3

언제로 변경하겠습니까?

09:58

12월 03일 화요일 10:07 (지각) -> 09:58 (출석) 수정 완료!

오늘은 12월 13일 금요일입니다. 기능을 선택해 주세요.

1. 출석 확인
 2. 출석 수정
 3. 크루별 출석 기록 확인
 4. 제적 위험자 확인
- Q. 종료
- 3

닉네임을 입력해 주세요.

빙티

이번 달 빙티의 출석 기록입니다.

12월 02일 월요일 13:00 (출석)
12월 03일 화요일 09:58 (출석)
12월 04일 수요일 10:02 (출석)
12월 05일 목요일 10:06 (지각)
12월 06일 금요일 10:01 (출석)
12월 09일 월요일 --:-- (결석)
12월 10일 화요일 10:08 (지각)

12월 11일 수요일 --:-- (결석)

12월 12일 목요일 --:-- (결석)

출석: 4회

지각: 2회

결석: 3회

면담 대상자입니다.

오늘은 12월 13일 금요일입니다. 기능을 선택해 주세요.

1. 출석 확인
 2. 출석 수정
 3. 크루별 출석 기록 확인
 4. 제적 위험자 확인
- Q. 종료
- 4

제적 위험자 조회 결과

- 빙티: 결석 3회, 지각 2회 (면담)
- 이든: 결석 2회, 지각 4회 (면담)
- 쿠키: 결석 2회, 지각 2회 (경고)
- 빙봉: 결석 1회, 지각 5회 (경고)

오늘은 12월 13일 금요일입니다. 기능을 선택해 주세요.

1. 출석 확인
 2. 출석 수정
 3. 크루별 출석 기록 확인
 4. 제적 위험자 확인
- Q. 종료
- Q

프로그래밍 요구 사항 1

- JDK 21 버전에서 실행 가능해야 한다.
- 프로그램 실행의 시작점은 `Application` 의 `main()` 이다.

- `build.gradle` 파일은 변경할 수 없으며, 제공된 라이브러리 이외의 외부 라이브러리는 사용하지 않는다.
- 프로그램 종료 시 `System.exit()` 를 호출하지 않는다.
- 프로그래밍 요구 사항에서 달리 명시하지 않는 한 파일, 패키지 등의 이름을 바꾸거나 이동하지 않는다.
- 자바 코드 컨벤션을 지키면서 프로그래밍한다.
 - 기본적으로 Java Style Guide를 원칙으로 한다.

프로그래밍 요구 사항 2

- indent(인덴트, 들여쓰기) depth를 3이 넘지 않도록 구현한다. 2까지만 허용한다.
 - 예를 들어 while문 안에 if문이 있으면 들여쓰기는 2이다.
 - 힌트: indent(인덴트, 들여쓰기) depth를 줄이는 좋은 방법은 함수(또는 메서드)를 분리하면 된다.
- 3항 연산자를 쓰지 않는다.
- 함수(또는 메서드)가 한 가지 일만 하도록 최대한 작게 만들어라.
- JUnit 5와 AssertJ를 이용하여 정리한 기능 목록이 정상적으로 작동하는지 테스트 코드로 확인한다.
 - 테스트 도구 사용법이 익숙하지 않다면 아래 문서를 참고하여 학습한 후 테스트를 구현한다.
 - JUnit 5 User Guide
 - AssertJ User Guide
 - AssertJ Exception Assertions
 - Guide to JUnit 5 Parameterized Tests

프로그래밍 요구 사항 3

- 함수(또는 메서드)의 길이가 15라인을 넘어가지 않도록 구현한다.
 - 함수(또는 메서드)가 한 가지 일만 잘 하도록 구현한다.
- else 예약어를 쓰지 않는다.

- else를 쓰지 말라고 하니 switch/case로 구현하는 경우가 있는데 switch/case도 허용하지 않는다.
- 힌트: if 조건절에서 값을 return하는 방식으로 구현하면 else를 사용하지 않아도 된다.
- Java Enum을 적용하여 프로그램을 구현한다.
- 구현한 기능에 대한 단위 테스트를 작성한다. 단, UI(System.out, System.in, Scanner) 로직은 제외한다.

라이브러리

- `camp.nextstep.edu.missionutils` 에서 제공하는 `DateTimes` 및 `Console API` 를 사용하여 구현해야 한다.
 - 현재 날짜와 시간을 가져오려면 `camp.nextstep.edu.missionutils.DateTimes` 의 `now()` 를 활용한다.
 - 사용자가 입력하는 값은 `camp.nextstep.edu.missionutils.Console` 의 `readLine()` 을 활용한다.

Copyright 우아한테크코스. All rights reserved.